

Week - 3 Module

Q1. Your task is to implement the Merge sort algorithm to sort the package values in increasing order. Merge sort follows a divide and conquer approach by dividing the dataset into smaller subarrays, sorting them recursively, and then merging them back together.

Sol.

```

import java.util.Scanner;

public class Main {
    static void merge (int arr[], int left, int mid, int right) {
        int n1 = mid - left + 1;
        int n2 = right - mid;
        int L[] = new int [n1];
        int R[] = new int [n2];
        for (int i=0; i<n1; i++) {
            L[i] = arr [left+i];
        }
        for (int j=0; j<n2; j++) {
            R[j] = arr [mid+1+j];
        }
        int i=0, j=0; k = left;
        while (i<n1 && j<n2) {
            if (L[i] <= R[j]) {
                arr[k] = L[i];
                i++;
            }
            else {
                arr[k] = R[j];
                j++;
            }
            k++;
        }
        while (i<n1) {
            arr[k] = L[i];
            i++; k++;
        }
    }
}

```

Date / /
Page No.

```

while (j < n2) {
    arr[k] = R[j];
    j++;
    k++;
}

```

```

{
    static void mergeSort (int arr[], int left, int right) {
        if (left < right) {
            int mid = left + (right - left) / 2;
            mergeSort (arr, left, mid);
            mergeSort (arr, mid + 1, right);
            merge (arr, left, mid, right);
        }
    }
}

```

```

public static void main (String [] args) {
    Scanner sc = new Scanner (System.in);
    int N = sc.nextInt();
    int arr[] = new int [N];
    for (int i = 0; i < N; i++) {
        arr[i] = sc.nextInt();
    }
    mergeSort (arr, 0, N-1);
    for (int i = 0; i < N; i++) {
        System.out.print(arr[i] + " ");
    }
    sc.close();
}
}

```

```

{

```

- Q2. Your task is to use MergeSort to arrange all processing times in ascending order. After sorting, perform following operations.
- 1> Display the sorted processing times
 - 2> Find the median processing time
 - 3> Count the number of orders whose processing time is greater than the median.
 - 4> Display the difference between the fastest & slowest process times.

Sol.

```

import java.util.*;
public class Main {
    static void mergeSort (int [] a, int l, int r) {
        if (l >= r) return;
        int m = (l+r)/2;
        mergeSort (a, l, m);
        mergeSort (a, m+1, r);
        int [] temp = new int [r-l+1];
        int i = l, j = m+1, k = 0;
        while (i <= m & j <= r)
            temp[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
        while (i <= m) temp[k++] = a[i++];
        while (j <= r) temp[k++] = a[j++];
        for (i = 0; i < temp.length; i++) {
            a[l+i] = temp[i];
        }
    }
    public static void main (String [] args) {
        Scanner sc = new Scanner (System.in);
        int n = sc.nextInt();
        int [] a = new int [n];
        for (int i = 0; i < n; i++) {
            a[i] = sc.nextInt();
        }
        mergeSort (a, 0, n-1);
    }
}

```


for (int x : a)

System.out.print(x + " ");

System.out.println();

int median = (n/2 == 1) ?

a[n/2] : (a[n/2-1] + a[n/2]) / 2;

int count = 0;

for (int x : a)

if (x > median) count++;

System.out.println("Median: " + median);

System.out.println("Orders Above Median: " + count);

System.out.println("Difference: " + (a[n-1] - a[0]));

{

}

Q3 Your task is to implement Quick sort to sort the response times in ascending order. built-in sorting methods are not allowed. After sorting, the operations team will use the report to identify high-latency servers and optimize system performance.

sol, import java.util.*;

public class Main {

static void quicksort (int a[], int low, int high) {

if (low < high) {

int p = partition (a, low, high);

quicksort (a, low, p-1);

quicksort (a, p+1, high);

}

}

static int partition (int a[], int low, int high) {

int pivot = a[high], i = low-1;

for (int j = low; j < high; j++) {

```

        if (a[i] < pivot) {
            i++;
            int t = a[i];
            a[i] = a[j];
            a[j] = t;
        }
    }

    int t = a[i+1];
    a[i+1] = a[high];
    a[high] = t;
    return i+1;
}

public static void main (String [] args) {
    Scanner sc = new Scanner (System.in);
    int n = sc.nextInt();
    int a[] = new int [n];
    for (int i = 0; i < n; i++) {
        a[i] = sc.nextInt();
    }
    quicksort (a, 0, n-1);
    for (int x : a) {
        System.out.print (x + " ");
    }
}

```

- Q4. Your task is to implement Quick sort to arrange the trade values from highest to lowest. After sorting, perform following operations
- 1) Display the sorted trade values
 - 2) Find top 5 highest trade values
 - 3) Calculate the average of the Top 5 values.
 - 4) Count how many trade values are greater than overall average of all trade values.

```

sol, import java.util.*;
public class Main {
    static void quickSort (int a[], int l, int h) {
        if (l < h) {
            int p = partition (a, l, h);
            quickSort (a, l, p-1);
            quickSort (a, p+1, h);
        }
    }

    static int partition (int a[], int l, int h) {
        int pivot = a[h], i = l-1;
        for (int j = l; j < h; j++) {
            if (a[j] > pivot) {
                i++;
                int t = a[i];
                a[i] = a[j];
                a[j] = t;
            }
        }
        int t = a[i+1];
        a[i+1] = a[h];
        a[h] = t;
        return i+1;
    }

    public static void main (String[] args) {
        Scanner sc = new Scanner (System.in);
        int n = sc.nextInt();
        int a[] = new int [n];
        long sum = 0;
        for (int i = 0; i < n; i++) {
            a[i] = sc.nextInt();
            sum += a[i];
        }
    }
}

```



```

quickSort(a, 0, n-1);
for (int x: a)
    System.out.print(x + " ");
System.out.println();
long top5 = 0;
System.out.print("Top 5: ");
for (int i=0; i<5; i++) {
    System.out.print(a[i] + " ");
    top5 += a[i];
}
System.out.println();
System.out.println("Average of Top 5: " + (top5/5));
double avg = (double) sum / n;
int count = 0;
for (int x: a) {
    if (x > avg) count++;
}
System.out.println("Values above Overall Average: "
    + count);

```

86
Your task is to implement Heap Sort and arrange all player scores in increasing order. The sorted output will help tournament officials generate ranking and determine qualification thresholds for future events.

lol
import java.util.*;
public class Main {
 static void heapify(int a[], int n, int i) {
 int largest = i, l = 2*i + 1, r = 2*i + 2;
 if (l < n && a[l] > a[largest]) largest = l;
 if (r < n && a[r] > a[largest]) largest = r;
 }
}

```
        if (largest != i) {
            int t = a[i];
            a[i] = a[largest];
            a[largest] = t;
            heapify(a, n, largest);
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int a[] = new int[n];
        for (int i = 0; i < n; i++)
            a[i] = sc.nextInt();
        for (int i = n/2 - 1; i >= 0; i--)
            heapify(a, n, i);
        for (int i = n - 1; i > 0; i--) {
            int t = a[0];
            a[0] = a[i];
            a[i] = t;
            heapify(a, i, 0);
        }
        for (int x : a)
            System.out.print(x + " ");
    }
}
```



```
9- import java.util.Scanner;
10
11- public class Main {
12
13-     // Function to merge two subarrays
14-     static void merge(int arr[], int left, int mid, int right) {
15-         int n1 = mid - left + 1;
16-         int n2 = right - mid;
17
18-         int L[] = new int[n1];
19-         int R[] = new int[n2];
20
21-         for (int i = 0; i < n1; i++)
22-             L[i] = arr[left + i];
23
24-         for (int j = 0; j < n2; j++)
25-             R[j] = arr[mid + 1 + j];
26
27-         int i = 0, j = 0, k = left;
28
29-         while (i < n1 && j < n2) {
30-             if (L[i] <= R[j]) {
31-                 arr[k] = L[i];
32-                 i++;
33-             } else {
34-                 arr[k] = R[j];
35-                 j++;
36-             }
37-             k++;

```

input

```
4
33 100 21 2
2 21 33 100
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

```

9- import java.util.*;
10
11- public class Main {
12
13-     static void mergeSort(int a[], int l, int r) {
14         if (l >= r) return;
15
16         int m = (l + r) / 2;
17         mergeSort(a, l, m);
18         mergeSort(a, m + 1, r);
19
20         int temp[] = new int[r - l + 1];
21         int i = l, j = m + 1, k = 0;
22
23         while (i <= m && j <= r)
24             temp[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
25
26         while (i <= m) temp[k++] = a[i++];
27         while (j <= r) temp[k++] = a[j++];
28
29         for (i = 0; i < temp.length; i++)
30             a[l + i] = temp[i];
31     }
32
33-     public static void main(String[] args) {

```

input

```

7
12 25 18 30 15 22 40
12 15 18 22 25 30 40
Median: 22
Orders Above Median: 3
Difference: 28

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```

```
9 import java.util.*;
10
11 public class Main {
12
13     static void quickSort(int a[], int low, int high) {
14         if (low < high) {
15             int p = partition(a, low, high);
16             quickSort(a, low, p - 1);
17             quickSort(a, p + 1, high);
18         }
19     }
20
21     static int partition(int a[], int low, int high) {
22         int pivot = a[high], i = low - 1;
23
24         for (int j = low; j < high; j++) {
25             if (a[j] < pivot) {
26                 i++;
27                 int t = a[i];
28                 a[i] = a[j];
29                 a[j] = t;
30             }
31         }
32
33         int t = a[i + 1];
34         a[i + 1] = a[high];
35         a[high] = t;
36     }
37 }
```

input

```
4
12 33 24 9
9 12 24 33
```

...Program finished with exit code 0
Press ENTER to exit console.


```
9 import java.util.*;
10
11 public class Main {
12
13     static void quickSort(int a[], int l, int h) {
14         if (l < h) {
15             int p = partition(a, l, h);
16             quickSort(a, l, p - 1);
17             quickSort(a, p + 1, h);
18         }
19     }
20
21     static int partition(int a[], int l, int h) {
22         int pivot = a[h], i = l - 1;
23
24         for (int j = l; j < h; j++) {
25             if (a[j] > pivot) { // Descending order
26                 i++;
27                 int t = a[i];
28                 a[i] = a[j];
29                 a[j] = t;
30             }
31         }
32
33         int t = a[i + 1];
```

input

```
6
500 1200 400 2000 1500 2500
2500 2000 1500 1200 500 400
Top 5: 2500 2000 1500 1200 500
Average of Top 5: 1540
Values Above Overall Average: 3
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

```
9 import java.util.*;
10
11 public class Main {
12
13     static void heapify(int a[], int n, int i) {
14         int largest = i;
15         int l = 2 * i + 1;
16         int r = 2 * i + 2;
17
18         if (l < n && a[l] > a[largest])
19             largest = l;
20
21         if (r < n && a[r] > a[largest])
22             largest = r;
23
24         if (largest != i) {
25             int t = a[i];
26             a[i] = a[largest];
27             a[largest] = t;
28             heapify(a, n, largest);
29         }
30     }
31
32     public static void main(String[] args) {
33         Scanner sc = new Scanner(System.in);
34
35         int n = sc.nextInt();
36         int a[] = new int[n];
37     }
```

input

```
4
200 100 240 300
100 200 240 300
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

```
9 import java.util.*;
10
11 public class Main {
12
13     static void heapify(int a[], int n, int i) {
14         int largest = i, l = 2 * i + 1, r = 2 * i + 2;
15
16         if (l < n && a[l] > a[largest]) largest = l;
17         if (r < n && a[r] > a[largest]) largest = r;
18
19         if (largest != i) {
20             int t = a[i];
21             a[i] = a[largest];
22             a[largest] = t;
23             heapify(a, n, largest);
24         }
25     }
26
27     public static void main(String[] args) {
28         Scanner sc = new Scanner(System.in);
29
30         int n = sc.nextInt();
31         int a[] = new int[n];
32         double sum = 0;
```

input

```
8
12 7 15 9 20 5 10 8
5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.50%
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```